

Recap

- data type conversion: int(), float(), str()
 - ✧ int('1.2') 
- data operators: ** ; * / % //; + -
 - ✧ if one operand is a float, the result is a float
 - ✧ +: → String concatenation; *: → String repetition, int*string
- variable
 - ✧ define a variable before using it.
 - ✧ multi-assignment, e.g. x,y,z = 1,2,3
- input(), print(), f-string.



Python

Introduction to Computer Programming

More about string and Decision structure

SCI-SHU 11

Fall 2026

Fang Yuan
fy12@nyu.edu

Lecture's Outline

- String
 - ✧ index and slicing
- Boolean logic
 - ✧ comparison operators (==, !=, >, >=, <, <=)
 - ✧ membership operators(in, not in)
 - ✧ logical operators (not, and, or)
- Decision structure
 - ✧ if statement
 - ✧ if-else statement
 - ✧ if-elif-else statement
 - ✧ nested if statement

More about string

ord() and chr()

- Computers use numbers to represent characters. The number is formally called a **Unicode**.

'A' --> 65

'a' --> 97

- ord()**: function returns the Unicode code for one-character string
- chr()**: function returns the character represented by the Unicode code

```
>>>ord('a')
97
>>>ord('A')
65
```

```
>>>chr(97)
'a'
>>>chr(65)
'A'
```

check ord(1), ord('12')

ord() and chr()

- Why are they useful? -We can do calculations with characters by converting them to numbers first.

Example: A Simple Secret Code:

To create a simple secret code, we shift each letter one position forward in the alphabet.

For example: $A \rightarrow B$; $B \rightarrow C$; $C \rightarrow D$

Write a program that asks the user to enter one uppercase letter and prints the encoded letter.

```
letter = input("Enter an uppercase letter (No Z): ")
encoded_letter = chr((ord(letter) + 1)
print("Encoded letter:", encoded_letter)
```

Length of string

len() function

- The len() function returns the length of a string- that is the number of characters it contains

```
>>>len("cat")
```

```
3
```

```
>>>len("")
```

```
0
```

```
>>>len("\n")
```

```
1
```

Indexing string

- An index tells us the position of a character in a string.

Positive Index	0	1	2	3	4	5	6	7	8	9	10	11
Negative Index	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
String	H	e	l	l	o		W	o	r	l	d	!

- Square brackets `[]` used to perform indexing into a string to get the value at a certain index.

```
s = "Hello World!"
```

```
>>> s[0]
'H'
>>> s[-12]
'H'
```

```
>>> s[11] # s[-1]
'!'
>>> s[7] # s[-5]
'o'
```

We can use indexing directly on a string

```
>>> "Hello World!"[0]
'H'
>>> "Hello World!"[-1]
'!'
```

Indexing string

- Index range

Positive index range: $0 \sim \text{len}(s) - 1$

Negative index range: $-\text{len}(s) \sim -1$

`s = "Hello World!"` `len(s)` is 12

- index range: $0 \sim 11$, $-12 \sim -1$

`s[12]` $\rightarrow$ raise an error
`s[-15]`

`IndexError: string index out of range`

Slicing string

- Slicing allows us to get part of a string.

```
string[start: stop: step]
```

- Get characters at **start**
- up to and including **stop – 1** (**stop** is not included)
- taking every **step** characters.
- If give two numbers, `string[start: stop]`, `step=1` by default
- You can also **omit** index and leave just colons
 - `string[start:]` → from start to the end
 - `string[:stop]` → from the beginning to stop (not included)
 - `string[:]` → the entire string
- You can mix positive and negative indices in slicing.

Slicing string

- Look at step first

positive step : means slicing from left to right

negative step : means slicing from right to left

	0	1	2	3	4	5	6	7	8	9	10	11
	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
s=	P	y	t	h	o	n	W	o	r	l	d	!

`s[3:7]` → 'honW', same as `s[3:7:1]`

`s[1:10:2]` → 'yhno!'

`s[:]` → 'PythonWorld!', same as `s[0:len(s):1]`

`s[::-1]` → '!dlroWnohtyP' same as `s[-1: -len(s)-1: -1]`

`s[2:]` → 'thonWorld!', same as `s[2:len(s):1]`

`s[:6]` → 'Python', same as `s[0:6:1]`

Slicing string

- Look at step first
positive step : means slicing from left to right
negative step : means slicing from right to left

	0	1	2	3	4	5	6	7	8	9	10	11
	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
s=	P	y	t	h	o	n	W	o	r	l	d	!

s[-9:7] → 'honW', same as s[3:7]

s[1:-2:2] → 'yhno!', same as s[1:10:2]

s[3:0] → "", empty string (not an error)

s[1:-1:-2] → "", empty string

s[-10:] → 'thonWorld!', same as s[2:len(s):1]

s[:-6] → 'Python', same as s[0:6:1]

You try it!

```
s = 'WelcomeICP'
```

```
>>> len(s)
```

```
>>> s[3]
```

```
>>> 'WelcomeICP'[-1]
```

```
>>> s[len(s)]
```

```
>>> s[-4]
```

```
>>> s[3:-2]
```

```
>>> s[2:-1:2]
```

```
>>> s[::-2]
```

```
>>> s[-4::-1]
```

```
>>> s[:7]
```

```
>>> s[2:-2:-3]
```

```
>>> s[7:1]
```

Immutable strings

- Strings are Immutable
Immutable means: once a string is created, its characters cannot be changed.

```
s = "Python"  
s[0] = "J"
```

✗ → gives an error

So, how can we change it? → Create a new string instead.

```
s = "Python"  
s = "J" + s[1:]  
print(s)
```

- Strings are immutable, we cannot change individual characters in an existing string. We can only create a new string.
- Slicing does not change the original string; it creates a **new string**.

Boolean Logic

Boolean Logic

bool type: True or False

comparison operators

logical operators

membership

return Boolean values

Comparison operators

- There are 6 comparison operators.
- These operators always return a bool (True or False)

Comparison operators	Meaning
<code>a == b</code>	is a equal to b?
<code>a != b</code>	is a different from b?
<code>a > b</code>	is a greater than b?
<code>a < b</code>	is a lower than b?
<code>a >= b</code>	is a greater than or equal to b?
<code>a <= b</code>	is a lower than or equal to b?

Comparison operators

Compare
numeric
values
(int or float)

a	b	Comparison operators	Result
1	2	<code>a == b</code>	False
1	2.0	<code>a != b</code>	True
1	2	<code>a > b</code>	False
1.0	2	<code>a < b</code>	True
1	2	<code>a >= b</code>	False
1	2.0	<code>a <= b</code>	True

- String comparison: compare their **Unicode!**

compares strings character by character from left to right until we find the **first different character**.

`'a' < 'b'` $\Rightarrow$ `97 < 98`

True

`'apple' < 'appreciate'`

True (`ord('l') < ord('r')`)

Comparison operators

```
>>> 8 > 'Apple'  
TypeError: '>=' not supported between instances  
of 'int' and 'str'
```

- Numeric types can be compared
(mixed numeric types int and float are allowed)
- Comparing a **string** with a **numeric value** will result in:
 - `==` will always return False
 - `!=` will always return True
 - `<`, `<=`, `>=` and `>` will raise a Type Error

You try it!

Evaluate the following comparison operations:

- `8 == 4.0`
- `'one' == 'two'`
- `5.1 >= 4.2`
- `5 == 5.0`
- `8.00001 != 8`
- `7.5 == '7.5'`
- `"abc" == "abcd"`
- `1 < int("12")`
- `'a' < 'b'`
- `'ab' < 'ac'`
- `'Word' < 'word'`
- `6 != 'six'`

Membership

The `in` or `not in` operators check whether a character or substring appear in a string.

`"tho" in "Python"` $\longrightarrow$ `True`

`"to" in "Python"` $\longrightarrow$ `False`

`"s" not in "Python"` $\longrightarrow$ `True`

Logic operators

a and b are both Boolean values

not a only one operand (to the right)
returns **True** if a is False

a or b two operands (one on each side)
returns **True** if at least one operand is True

a and b two operands (one on each side)
returns **True** if both operands are True

Logic operators

Truth Tables

<i>a</i>	<i>not a</i>
False	True
True	False

<i>and</i>	False	True
False	False	False
True	False	True

<i>or</i>	False	True
False	False	True
True	True	True

Chaining Operators

- Numeric/String operators can be combined with Comparison and Logical operators to create complex **Boolean** expressions.

Example:

```
100 > 1 and -10 ** 2 <= -100 or "foo" == "foo" + "bar"
```

what order will the operations be evaluated in?

Operator Precedence

Operators	Meaning
()	parentheses are evaluated first
**	exponentiation
+, -	unary + and - (signs)
*, /, //, %	multiplication, division(s) and modulo
+, -	addition and subtraction
==, !=, <, <=, >=, >, in, not in	comparison operators
not	
and	
or	

NOTE: Even if your expression follows operator precedence, it is good practice to use **parentheses**, at the very least, for readability

Operator Precedence

`100 > 1 and -10 ** 2 <= -100 or "foo" == "foo" + "bar"`

Arithmetic operators

`100 > 1 and -100 <= -100 or "foo" == "foobar"`



Comparison

`True and True or False`



Logic operators (not → and → or)

`True or False`

`True`

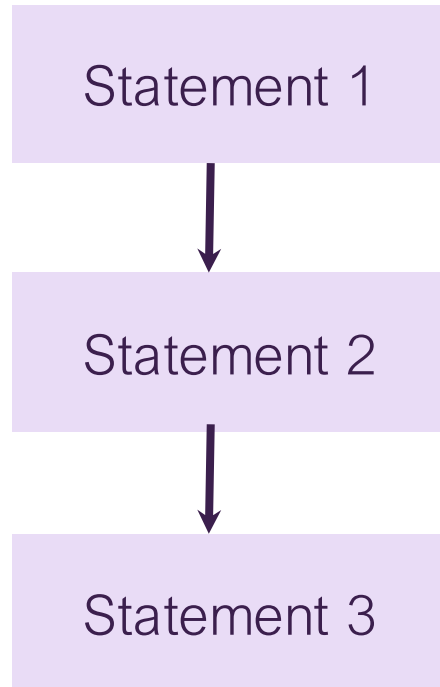
Hard to read!

Use parentheses !

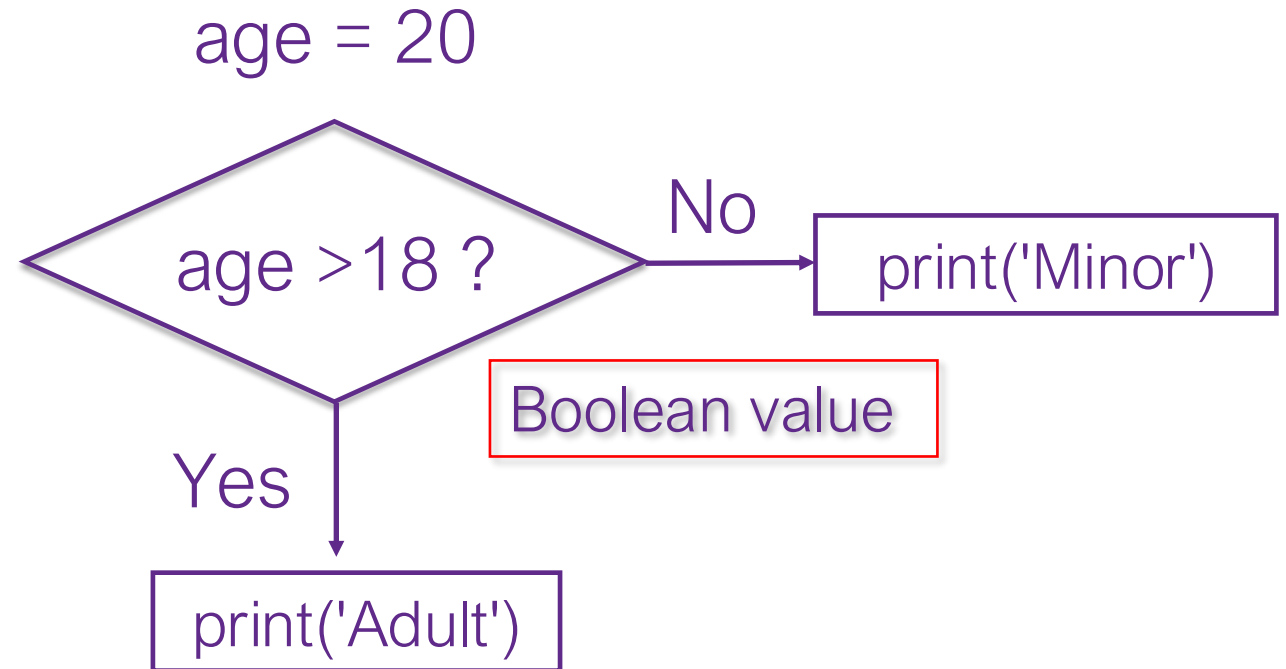
Decision Structures

Decision structure

- So far, our programs execute statements one by one, from top to bottom.
- Sometimes, we want to execute different code under different conditions.



.....
↓



A decision structure allows a program to choose what to do based on a condition.

if statement

```
if <condition>:  
    statement1  
    statement2  
    .....  
outside_statement
```

- <condition>: Boolean expression with value True or False
- **Indentation** matters! Statements in the same block must have the same indentation.
- Run code within if block if condition is True
- once the indentation goes back one level, the body of the if-statement ends

if statement

Example:

```
number = int(input('Enter a number: '))  
if number >= 0:  
    print(f"{number} is positive.")  
print("That's all folks.")
```

number = 2

Test if condition, it's True, run if block

run this line after the if block is finished

```
Enter a number: 2  
2 is positive
```

if statement

Example:

```
number = int(input('Enter a number: '))  
if number >= 0:  
    print(f"{number} is positive.")  
print("That's all folks.")
```

number = -1

Test if condition, it's False, skip if block

run this line

```
Enter a number: -1  
That's all folks.
```

if-else statement

```
if <condition>:  
    statement  
else:  
    statement  
.....
```

Check the condition:

- True → execute the if block
- False → execute the else block

if-else statement

Example:

```
number = int(input('Enter a number: '))
```

number = 2

```
if number >= 0:  
    print(f"{number} is positive.")
```

Condition is True,
run if block

```
else:  
    print(f"{number} is strictly negative.")
```

skip else block

```
print("That's all folks.")
```

Run this line after the
if-else block is finished.

Enter a number: 2

2 is positive

That's all folks.

if-else statement

Example:

```
number = int(input('Enter a number: '))
```

number = -2

```
if number >= 0:
```

Condition is False,
skip if block,

```
    print(f"{number} is positive.")
```

```
else:
```

```
    print(f"{number} is strictly negative.")
```

run else block

```
print("That's all folks.")
```

Run this line after the
if-else block is finished.

Enter a number: -2

-2 is strictly negative

That's all folks.

You try it!

Programming Exercise: Username Validator

A website requires a username to satisfy both of the following conditions:

- The username must contain at least 6 characters.
- The username must contain "_".

Write a program that asks the user to enter a username.

- If both conditions are satisfied, print: `Valid username`
- Otherwise, print: `Invalid username`

if-elif-else statement

```
if <condition>:  
    statement  
elif <condition>:  
    statement  
elif .....  
  
else:  
    statement  
.....
```

- if-elif-else allows us to choose among multiple alternatives.
- Checks the conditions from top to bottom and run the **first** block whose condition is True.
- use as many elif-statements as needed. . .
- use else-statement at the end if needed.
- If none of the conditions is True the body of the else clause will be executed (if present)

Once a condition is True, execute that block and then exit the entire if-elif-else structure.

if-elif-else statement

Example: Write a program that prompts the user for an integer:

- print "Positive" if the number is positive
- print "Negative" if the number is negative
- print "Zero" if the number is 0

```
number = int(input('Enter a number: '))
if number > 0:
    print(f"{number} is positive.")
elif number < 0:
    print(f"{number} is negative.")
else: # number == 0
    print(f"{number} is zero.")
print("This is the end of the program.")
```

if-elif-else statement

```
number = int(input('Enter a number: '))
```

number = 2

```
if number > 0:
```

Condition is True, run this if block

```
    print(f"{number} is positive.")
```

```
elif number < 0:
```

```
    print(f"{number} is negative.")
```

```
else: # number == 0
```

```
    print(f"{number} is zero.")
```

skip the remaining elif-else block

```
print("This is the end.")
```

run this line when if-elif-else block is finished

```
Enter a number: 2
```

```
2 is positive
```

```
This is the end.
```

if-elif-else statement

```
number = int(input('Enter a number: '))
```

number = -4

```
if number > 0:
```

Condition is False, skip it, test the next elif condition.

```
    print(f"{number} is positive.")
```

```
elif number < 0:
```

It's True, run elif block, skip the remaining else block

```
    print(f"{number} is negative.")
```

```
else: # number == 0
```

```
    print(f"{number} is zero.")
```

```
print("This is the end.")
```

run this line when if-elif-else block is finished

```
Enter a number: -4
```

```
-4 is negative
```

```
This is the end.
```

if-elif-else statement

```
number = int(input('Enter a number: '))
```

number = 0

```
if number > 0:
```

Condition is False, skip it, test the next elif condition.

```
    print(f"{number} is positive.")
```

```
elif number < 0:
```

It's False, skip it

```
    print(f"{number} is negative.")
```

```
else: # number == 0
```

```
    print(f"{number} is zero.")
```

directly run else block

```
print("This is the end.")
```

run this line when if-elif-else block is finished

```
Enter a number: 0
```

```
0 is zero
```

```
This is the end.
```

if-elif-else statement

Example: Ask for the user to enter a season, print out:

Season	Summer	Winter	Fall	Spring
Output	Sun	Snow	Leaves	Flowers

```
season = input("Enter a season: ")
if season == "Summer":
    print("Sun")
elif season == "Winter":
    print("Snow")
elif season == "Fall":
    print("Leaves")
else: # season is Spring
    print("Flowers")
```

nested if-elif-else statement

```
age = 20
has_ticket = "yes"
if age >= 18:
    if has_ticket == "yes":
        print("You may enter.")
    elif has_ticket == "no":
        print("Please buy a ticket.")
    else:
        print("Invalid ticket information.")
else:
    print("You are too young to enter.")
```

- A decision structure placed inside another decision structure.
- Indentation shows which if, elif, and else belong together.

Compare the following two programs!

```
number = int(input('Enter a number: '))
if number > 0:
    print(f"{number} is positive.")
elif number < 0:
    print(f"{number} is negative.")
else: # number == 0
    print(f"{number} is zero.")
print("This is the end of the program.")
```

```
number = int(input('Enter a number: '))
if number > 0:
    print(f"{number} is positive.")
if number < 0:
    print(f"{number} is negative.")
if number == 0:
    print(f"{number} is zero.")
print("This is the end of the program.")
```

Both programs produce the same output! What's the difference?

if-elif-else: Clearly shows that only one case will happen.

Once a condition is True, stops checking the remaining conditions.

Multiple if statements:

- Each if condition is an independent decision.
- checks every if condition, even if an earlier condition is already True.

Use if-elif-else when the cases are **mutually exclusive**.

```
score = int(input("Enter a score: "))
if score >= 90:
    print("Excellent")
elif score >= 80:
    print("Good")
elif score >= 60:
    print("Pass")
else:
    print("Fail")
print("This is the end.")
```

```
Enter a score: 87

Good
```

```
Enter a score: 87

Good

Pass
```

```
score = int(input("Enter a score: "))
if score >= 90:
    print("Excellent")
if score >= 80:
    print("Good")
if score >= 60:
    print("Pass")
if score < 60:
    print("Fail")
print("This is the end.")
```

Enter 93? OK!

Each if is an independent decision.
More than one block can be executed.

You try it!

Programming Exercise: Next Seat

- A seat code consists of a row letter followed by a seat number, such as: C12
- Ask the user to enter a seat code. If the row letter is between "A" and "Y", print the same seat number **in the next row**.
- For example:

```
Enter a seat: C12
Next row seat: D12
```
- If the row is "Z", print: This is the last row.
- Otherwise, print: Invalid seat code.

You try it!

Programming Exercise: Movie Ticket System

A movie theater determines the ticket price based on the customer's age and membership status. Write a program that asks the user to enter:

- Their age and Whether they are a member ("yes" or "no")
- The ticket price is determined as follows:

If the customer is under 18:

Member → 30 RMB

Non-member → 40 RMB

If the customer is 18–59:

Member → 50 RMB

Non-member → 70 RMB

If the customer is 60 or older:

Member → 30 RMB

Non-member → 35 RMB

After determining the price, print the ticket price.

Example output:

```
Enter your age: 21
Are you a member? yes
Ticket price: 50 RMB
```

```
Enter your age: 65
Are you a member? no
Ticket price: 35 RMB
```

Reading and Assignment

- This week readings
 - Read chapter 3 and 4 in the textbook
- This week recitation
 - recitation 03.
- Homework assignment 2 this week.
 - submit in Gradescope